

2026-09-24

A documentation linter for R packages

Nic Crane
NC Data Labs Ltd

This template was last revised on 2026-03-24.

Executive Summary

R has some excellent tooling for code styling and linting, but there are gaps when thinking about consistency in the documentation and prose within R packages. Hard-to-read documentation costs users time, puts off newcomers, and is becoming more common as more package code and documentation is written by AI tools.

Vale is the established tool for this job in many other ecosystems. It is a prose linter with rules defined as data, a library of published styles, and integrations for editors and CI. It can read Markdown and pull the text out of comments in R files, but that is as far as its knowledge of R goes. Its default styles report R function and argument names as spelling mistakes, it has no sense of how an R package is laid out, and it has to be installed and configured by hand outside of R.

This project fills that gap with an R package that installs and runs Vale, and a Vale style built for R packages. Together they lint vignettes, READMEs and the comments in R source files, including the roxygen comments that become help pages, from the R console, the command line or a CI job. The style is assembled from Vale's existing rules with thresholds set by running them across real CRAN packages and a vocabulary of R terms so the defaults are quiet out of the box.

The deliverables are the package on CRAN, the style published for anyone using Vale directly, and a GitHub Action, over two milestones ending in February 2027, for a total of \$4,500.

Signatories

Project team

- [Nic Crane](#): R developer with over 10 years of experience, one of the maintainers of the [arrow](#) R package and contributor to [ellmer](#).

Contributors

Consulted

- [Etienne Bacher](#): independent developer, author of [jarl](#), a fast linter for R funded by the ISC in 2025.

The Problem

When working in R, it's already common to have standard conventions for code style: packages like [lintr](#) ([Hester et al. 2025](#)) and tools like [air](#) and [jarl](#) are widely used to ensure that code conforms to agreed standards. Documentation doesn't receive the same attention. While R CMD `check` verifies that documentation exists and matches usage, there is no common tooling for ensuring that docs are readable by a human, whether that is the help pages, vignettes and READMEs read by users or the code comments read by contributors, and this responsibility falls to authors and reviewers.

Documentation that is hard to read, whether because of dense sentences or a tone that assumes things are obvious, increases cognitive load ([Sweller 1988](#)), frustrates users, and can be a real barrier to adoption.

Organisations like rOpenSci recognise this, and their software review process asks reviewers to judge whether documentation is clear and sufficient ([rOpenSci et al. 2026](#)). The problem is also growing: R packages are increasingly written with the help of, or exclusively by, AI tools, and without oversight it's easy to end up with verbose documentation.

Some prior work exists. The [docreview](#) package, on CRAN between 2021 and 2024, contained 5 checks including readability scores, but stalled due to the complexity of its architecture. The [spelling](#) package checks words but not prose. Outside of R, [Vale](#) ([Kato 2026](#)) is a mature documentation linter with prose rules and packaged style guides, which runs in editors and on CI. It handles Markdown and can extract the text of comments from R source files, but it has no knowledge of R package structure or R vocabulary, its built-in styles flag function and argument names as spelling errors, and using it requires installing and configuring a separate binary outside of R.

The R ecosystem lacks an opinionated documentation linter, usable from R, that helps both humans and AI tools write better docs for R packages.

The proposal

Overview

This proposal is for an R package which wraps Vale, together with a Vale style written for R package documentation, so that documentation can be linted from R, on the command line and on CI. It lints both the external documentation of a package, its vignettes and READMEs, and the comments in its R source files, including the roxygen comments from which help pages are generated, and reports the file, line, rule and a message explaining what was wrong, as a data frame in R or as JSON for AI tools and CI reporting.

Vale already provides rules for sentence length, readability scores, and words often reported by readers as condescending like “simply”, “just” and “obviously”, spread across several existing styles and tuned for general prose. The work in this project is to select the subset of those rules that make sense for R package documentation, set their thresholds by running them over real packages so that they aren't noisy out of the box, and add a vocabulary of common R terms. Together these form

a default style that ships with the package and is also published for people who use Vale directly. Rules are configured from a Vale config file at the package root, which the package can generate, so that users can adjust the defaults according to taste.

Building on Vale keeps the amount of novel code small. Vale handles extracting comments from R files, parsing Markdown, running rules and integrating with editors, and its rules are defined as data rather than code, so the community can contribute rules without needing to write R. The package itself has three parts: locating an existing Vale binary or downloading one for the user's platform when asked, following the pattern used by blogdown and tinytex; the style; and the R interface, config generation and GitHub Action.

The main dependency is Vale itself, which is actively maintained and publishes binaries for Linux, macOS and Windows on every release. Nothing requires a service or a paid account, so there are no ongoing costs.

Project plan

Start-up phase

The project will be developed in a public GitHub repository under the MIT licence, with issues used to track all work.

Technical delivery

Initial experiments running Vale over an existing R package confirm that it extracts comments from R source files with correct line numbers and handles vignettes and READMEs without any additional work. They also show that its built-in styles are noisy on R code, with most reported problems being function and argument names flagged as spelling errors, which is what an R-specific style and vocabulary will address.

Milestone 1 - 31st January 2027: working package and first rules

Consultation with package maintainers and a review of documentation feedback in public software review threads, such as rOpenSci's, to decide which rules will be most useful in the default style. The code delivered is the R package which installs or locates Vale, runs it over a package, and returns lints as a data frame, with an initial set of rules and snapshot tests, working on Linux, macOS and Windows.

Milestone 2 - 28th February 2027: default style, release and GitHub Action

The remaining rules, the vocabulary of R terms, config file generation and JSON output, each rule documented with examples of passes and failures. A run over the 100 most downloaded CRAN packages drives tuning of the rules and defaults. The package is submitted to CRAN, the style is published so that it can be installed directly by Vale users, and a GitHub Action and pre-commit hook are released so that linting can run on CI, together with a command that writes the active rules into a project's AGENTS.md file. A post on the R Consortium blog reports on the project.

Other aspects

All code is MIT licensed and developed in the open on GitHub. The package will be submitted to CRAN and the style published on GitHub so that it can be used with Vale directly. The work will

be publicised with a post on the R Consortium blog, posts on LinkedIn, Mastodon and Bluesky at each milestone, and submission to R Weekly.

Budget & funding plan

The budget is labour only, at USD 90 per hour, covering 50 hours of part-time work over two months. There are no other costs, as the project uses GitHub for hosting, CI and releases.

Milestone	Target	Hours	USD
1: Working package and first rules	31 Jan 2027	30	2,700
2: Default style, release and GitHub Action	28 Feb 2027	20	1,800
Total		50	4,500

Success

Definition of done

The project is complete when:

- the package is published on CRAN and installs or locates Vale on Linux, macOS and Windows
- it lints R code comments, vignettes and READMEs using a default style that ships with the package, with rules configurable from a file at the package root
- the style is also published so that it can be installed and used directly by Vale users
- the defaults have been tuned so that the tool runs without error on the 100 most downloaded CRAN packages
- the GitHub Action, pre-commit hook and AGENTS.md command are released and documented

Measuring success

During the project, progress is tracked by milestone completion against the dates in the project plan, with work visible in public GitHub issues. After release, the main indicators are the number of packages using the GitHub Action, found via GitHub code search, and rules or vocabulary contributed by people outside the project. Completion of the grant does not depend on adoption, since uptake of developer tooling takes longer than the grant period.

Future work

- Roxygen-aware linting, so that rules can be applied per help topic and example code is not treated as prose
- Linting of error and warning messages
- Additional named styles, so that organisations with a house style can publish their own rule sets for R packages

Hester, Jim, Florent Angly, Michael Chirico, et al. 2025. “Static Code Analysis for R.” *Journal of Open Source Software* 10 (108): 7240. <https://doi.org/10.21105/joss.07240>.

- Kato, Joseph. 2026. *Vale: A Linter for Prose*. <https://vale.sh>.
- rOpenSci, Brooke Anderson, Scott Chamberlain, et al. 2026. *rOpenSci Packages: Development, Maintenance, and Peer Review*. V. v1.0.0. Zenodo, released March. <https://doi.org/10.5281/zenodo.18885936>.
- Sweller, John. 1988. “Cognitive Load During Problem Solving: Effects on Learning.” *Cognitive Science* 12 (2): 257–85. https://doi.org/10.1207/s15516709cog1202_4.